



(CSE 4802: Compiler Design Lab)

LAB 6

Report on Implementation of an HTML Structure Validator for Basic Web Pages Using LEX and YACC

Submitted by:

Md. Mosharrifur Rahman Ratul

Student ID: 210041213

Section: 2A

BSc in CSE, Dept of Computer Science and Engineering
Islamic University of Technology (IUT)

August 15, 2026

Contents

1	Introduction	2
2	Problem Description	2
3	Code Implementation	2
4	Input and Output	9

1 Introduction

HTML syntax validation is an important part of processing web documents to ensure that the structure of an HTML document follows the expected syntax and nesting rules. In this laboratory, LEX (Flex) is used to perform lexical analysis of HTML tags and symbols, while YACC (Bison) is used to parse the resulting tokens according to a defined grammar. Semantic actions in YACC are used to identify whether the given HTML input is syntactically valid or contains errors.

2 Problem Description

The objective of this laboratory is to develop an HTML syntax validator using LEX and YACC. The LEX analyzer identifies HTML elements such as opening tags, closing tags, attributes, and other relevant symbols and passes them as tokens to the YACC parser. The YACC parser then checks whether the sequence of tokens follows the defined HTML grammar, including the proper nesting and matching of opening and closing tags.

The validator should be able to distinguish between syntactically correct and incorrect HTML inputs and generate an appropriate validation message as output. Three sample HTML inputs are considered to demonstrate the behavior of the validator for both valid and invalid cases.

3 Code Implementation

The implementation consists of two source files: the LEX source file `validator.l`, which performs lexical analysis, and the YACC source file `validator.y`, which performs syntax analysis and validation.

LEX Source File: `lexer.l`

```
1  
2 %  
3 #include "parser.tab.h"
```

```

4  #include <string.h>
5  #include <stdlib.h>
6  %}
7
8  %option noyywrap
9
10 %%
11
12 "<html>"           { return HTML_OPEN; }
13 "</html>"          { return HTML_CLOSE; }
14
15 "<head>"           { return HEAD_OPEN; }
16 "</head>"          { return HEAD_CLOSE; }
17
18 "<title>"          { return TITLE_OPEN; }
19 "</title>"         { return TITLE_CLOSE; }
20
21 "<body>"           { return BODY_OPEN; }
22 "</body>"          { return BODY_CLOSE; }
23
24 "<h1>"             { return H1_OPEN; }
25 "</h1>"            { return H1_CLOSE; }
26
27 "<h2>"             { return H2_OPEN; }
28 "</h2>"            { return H2_CLOSE; }
29
30 "<p>"              { return P_OPEN; }
31 "</p>"             { return P_CLOSE; }
32
33 "<div>"            { return DIV_OPEN; }
34 "</div>"           { return DIV_CLOSE; }
35
36 "<b>"              { return B_OPEN; }
37 "</b>"             { return B_CLOSE; }
38
39 "<i>"              { return I_OPEN; }
40 "</i>"             { return I_CLOSE; }
41
42 "[^<>\n]+"         {
43                       yylval.str = strdup(yytext);
44                       return TEXT;
45                       }
46
47 "[ \t\r\n]+"       ;
48
49 "."                 {
50                       return yytext[0];
51                       }
52
53 %%

```

YACC Source File: **validator.y**

```
1
2 %{
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6
7 extern FILE *yyin;
8 extern int yylex();
9
10 FILE *outFile;
11
12 void yyerror(const char *s);
13 %}
14
15 %define parse.error verbose
16
17 %union {
18     char *str;
19 }
20
21 %token HTML_OPEN HTML_CLOSE
22 %token HEAD_OPEN HEAD_CLOSE
23 %token TITLE_OPEN TITLE_CLOSE
24 %token BODY_OPEN BODY_CLOSE
25 %token H1_OPEN H1_CLOSE
26 %token H2_OPEN H2_CLOSE
27 %token P_OPEN P_CLOSE
28 %token DIV_OPEN DIV_CLOSE
29 %token B_OPEN B_CLOSE
30 %token I_OPEN I_CLOSE
31
32 %token <str> TEXT
33
34 %%
35
36 document
37     : HTML_OPEN head body HTML_CLOSE
38     ;
39
40 head
41     : HEAD_OPEN title HEAD_CLOSE
42     ;
43
44 title
```

```

45 : TITLE_OPEN text TITLE_CLOSE
46 ;
47
48 body
49 : BODY_OPEN body_content BODY_CLOSE
50 ;
51
52 body_content
53 : /* empty */
54 | body_content element
55 ;
56
57 element
58 : heading1
59 | heading2
60 | paragraph
61 | division
62 | bold
63 | italic
64 | text
65 ;
66
67 /* H1 */
68
69 heading1
70 : H1_OPEN body_content H1_CLOSE
71 | H1_OPEN body_content H2_CLOSE
72 {
73     fprintf(outFile,
74         "Error: <h1> is closed incorrectly.\n");
75     fprintf(outFile,
76         "Expected: </h1>\n");
77     fprintf(outFile,
78         "Found: </h2>\n");
79     YYERROR;
80 }
81 ;
82
83 /* H2 */
84
85 heading2
86 : H2_OPEN body_content H2_CLOSE
87 | H2_OPEN body_content H1_CLOSE
88 {
89     fprintf(outFile,
90         "Error: <h2> is closed incorrectly.\n");
91     fprintf(outFile,
92         "Expected: </h2>\n");
93     fprintf(outFile,
94         "Found: </h1>\n");

```

```

95         YYERROR;
96     }
97     ;
98
99     /* Paragraph */
100
101     paragraph
102     : P_OPEN body_content P_CLOSE
103     | P_OPEN body_content BODY_CLOSE
104     {
105         fprintf(outFile,
106             "Error: <p> is not closed correctly.\n");
107         fprintf(outFile,
108             "Expected: </p>\n");
109         fprintf(outFile,
110             "Found: </body>\n");
111         YYERROR;
112     }
113     ;
114
115     /* Division */
116
117     division
118     : DIV_OPEN body_content DIV_CLOSE
119     | DIV_OPEN body_content BODY_CLOSE
120     {
121         fprintf(outFile,
122             "Error: <div> is not closed correctly.\n");
123         fprintf(outFile,
124             "Expected: </div>\n");
125         fprintf(outFile,
126             "Found: </body>\n");
127         YYERROR;
128     }
129     ;
130
131     /* Bold */
132
133     bold
134     : B_OPEN body_content B_CLOSE
135     | B_OPEN body_content P_CLOSE
136     {
137         fprintf(outFile,
138             "Error: <b> is not closed correctly.\n");
139         fprintf(outFile,
140             "Expected: </b>\n");
141         fprintf(outFile,
142             "Found: </p>\n");
143         YYERROR;
144     }

```

```

145     ;
146
147     /* Italic */
148
149     italic
150     : I_OPEN body_content I_CLOSE
151     | I_OPEN body_content P_CLOSE
152     {
153         fprintf(outFile,
154             "Error: <i> is not closed correctly.\n");
155         fprintf(outFile,
156             "Expected: </i>\n");
157         fprintf(outFile,
158             "Found: </p>\n");
159         YYERROR;
160     }
161     ;
162
163     /* Text */
164
165     text
166     : TEXT
167     ;
168
169     %%
170
171     void yyerror(const char *s)
172     {
173         fprintf(outFile, "Syntax Error: %s\n", s);
174
175         /*
176          * General error messages.
177          *
178          * These are used when Bison encounters something
179          * that is not covered by our specific error rules.
180          */
181
182         if (strstr(s, "unexpected HTML_CLOSE"))
183         {
184             fprintf(outFile,
185                 "Expected: a valid HTML structure before </html>.\n");
186         }
187         else if (strstr(s, "unexpected HEAD_CLOSE"))
188         {
189             fprintf(outFile,
190                 "Expected: </title> before </head>.\n");
191         }
192         else if (strstr(s, "unexpected TITLE_CLOSE"))
193         {

```



```

194         fprintf(outFile,
195                 "Expected: valid title content before </title
                >.\n");
196     }
197     else if (strstr(s, "unexpected BODY_CLOSE"))
198     {
199         fprintf(outFile,
200                 "Expected: a valid body element before </body
                >.\n");
201     }
202     else if (strstr(s, "unexpected H1_CLOSE"))
203     {
204         fprintf(outFile,
205                 "Expected: </h1>.\n");
206     }
207     else if (strstr(s, "unexpected H2_CLOSE"))
208     {
209         fprintf(outFile,
210                 "Expected: </h2>.\n");
211     }
212     else if (strstr(s, "unexpected P_CLOSE"))
213     {
214         fprintf(outFile,
215                 "Expected: </p>.\n");
216     }
217     else if (strstr(s, "unexpected DIV_CLOSE"))
218     {
219         fprintf(outFile,
220                 "Expected: </div>.\n");
221     }
222     else if (strstr(s, "unexpected B_CLOSE"))
223     {
224         fprintf(outFile,
225                 "Expected: </b>.\n");
226     }
227     else if (strstr(s, "unexpected I_CLOSE"))
228     {
229         fprintf(outFile,
230                 "Expected: </i>.\n");
231     }
232 }
233
234 int main()
235 {
236     int result;
237
238     yyin = fopen("input3.html", "r");
239
240     if (yyin == NULL)
241     {

```

```

242     printf("Cannot open input3.html\n");
243     return 1;
244 }
245
246 outFile = fopen("output.txt", "w");
247
248 if (outFile == NULL)
249 {
250     printf("Cannot create output.txt\n");
251     fclose(yyin);
252     return 1;
253 }
254
255 result = yyparse();
256
257 if (result == 0)
258 {
259     fprintf(outFile,
260             "HTML document is structurally valid.\n");
261 }
262 else
263 {
264     fprintf(outFile,
265             "HTML document contains syntax errors.\n");
266 }
267
268 fclose(yyin);
269 fclose(outFile);
270
271 return 0;
272 }

```

4 Input and Output

The following examples demonstrate the validation of different HTML inputs. Each input is paired with its corresponding output so that the behavior of the validator can be observed clearly. It is tested with both correct and wrong html syntax.

Case 1: Valid HTML Input

The first test case contains a syntactically correct HTML document. The opening and closing tags are properly matched and nested.

Input: input1.html

```
1
2 <html>
3 <head>
4 <title>My Website</title>
5 </head>
6
7 <body>
8 <h1>Hello World</h1>
9 <h2>Welcome</h2>
10
11 <p>This is a <b>simple</b> paragraph.</p>
12
13 <div>
14 <p>This is inside a div.</p>
15 <i>Italic text</i>
16 </div>
17
18 </body>
19 </html>
```

Output: output1.txt

```
1
2 The HTML document is structurally valid.
```

Case 2: Invalid HTML Input

The second test case contains an invalid HTML structure. The validator detects the syntax error according to the grammar defined in the YACC parser.

Input: input2.html

```
1
2 <html>
3 <head>
4 <title>My Website</title>
5 </head>
6
7 <body>
8 <h1>Hello</h1>
9 <p>This is a paragraph.
10 </body>
```

```
11 | </html>
```

Output: output2.txt

```
1 Error: <h1> is closed incorrectly.  
2 Expected: </p>  
3 Found: </body>  
4 HTML document contains syntax errors.
```

Case 3: Invalid HTML Input

The third test case demonstrates another invalid HTML structure. The generated output shows the corresponding validation result produced by the LEX/YACC program.

Input: input3.html

```
1 <html>  
2 <head>  
3 <title>My Website</title>  
4 </head>  
5  
6 <body>  
7 <h1>Hello</h2>  
8 </body>  
9 </html>
```

Output: output3.txt

```
1 Error: <h1> is closed incorrectly.  
2 Expected: </h1>  
3 Found: </h2>  
4 HTML document contains syntax errors.
```